

Manual Completo de Classes em Java

Índice

1. [Introdução às Classes](#)
2. [Estrutura Básica de uma Classe](#)
3. [Atributos](#)
4. [Métodos](#)
5. [Construtores](#)
6. [Modificadores de Acesso](#)
7. [Encapsulamento](#)
8. [Exemplos Práticos](#)
9. [Exercícios](#)

Introdução às Classes

O que é uma Classe?

Uma **classe** em Java é um modelo ou blueprint que define a estrutura e comportamento dos objetos. Ela encapsula:

- **Atributos:** características ou propriedades
- **Métodos:** comportamentos ou ações

Analogia Prática

Pense em uma classe como uma **forma de bolo** e os objetos como **bolos** feitos com essa forma.

```
// Forma (Classe)
class Bolo {
    // Atributos: características do bolo
    String sabor;
    double peso;

    // Métodos: ações do bolo
    void assar() { ... }
    void decorar() { ... }
}

// Bolos (Objetos)
Bolo boloChocolate = new Bolo();
Bolo boloMorango = new Bolo();
```

Estrutura Básica de uma Classe

```
// Sintaxe geral
[modificador] class NomeDaClasse {
    // ATRIBUTOS (variáveis)

    // CONSTRUTORES

    // MÉTODOS
}
```

Exemplo Básico

```
public class Carro {
    // Atributos
    String marca;
    String modelo;
    int ano;
    double velocidade;

    // Construtor
    public Carro(String marca, String modelo, int ano) {
        this.marca = marca;
        this.modelo = modelo;
        this.ano = ano;
        this.velocidade = 0;
    }

    // Métodos
    public void acelerar(double incremento) {
        this.velocidade += incremento;
    }

    public void frear(double decremento) {
        this.velocidade -= decremento;
    }
}
```

Atributos

Definição

Atributos são **variáveis** que armazenam o estado de um objeto.

Tipos de Atributos

1. Atributos de Instância

```
public class Pessoa {  
    // Atributos de instância (cada objeto tem sua própria cópia)  
    String nome;  
    int idade;  
    double altura;  
}
```

2. Atributos de Classe (static)

```
public class Contador {  
    // Atributo de classe (compartilhado por todos os objetos)  
    static int totalContadores = 0;  
    int valor;  
  
    public Contador() {  
        totalContadores++; // Incrementa o contador global  
    }  
}
```

3. Atributos Constants (final)

```
public class Circulo {  
    // Atributo constante (não pode ser alterado)  
    final double PI = 3.14159;  
    double raio;  
}
```

Exemplo Completo

```
public class Estudante {  
    // Atributos  
    private String nome;  
    private int matricula;  
    private double nota1;  
    private double nota2;  
    private static int totalEstudantes = 0;  
    final String ESCOLA = "Universidade Java";  
  
    // Construtor  
    public Estudante(String nome, int matricula) {  
        this.nome = nome;  
        this.matricula = matricula;  
        totalEstudantes++;  
    }  
}
```

Métodos

Definição

Métodos são **funções** que definem o comportamento dos objetos.

Tipos de Métodos

1. Métodos de Instância

```
public class Calculadora {  
    // Método de instância  
    public double somar(double a, double b) {  
        return a + b;  
    }  
  
    public void imprimirResultado(double resultado) {  
        System.out.println("Resultado: " + resultado);  
    }  
}
```

2. Métodos de Classe (static)

```
public class Matematica {  
    // Método estático (não precisa de objeto)  
    public static double calcularAreaCirculo(double raio) {  
        return Math.PI * raio * raio;  
    }  
}  
  
// Uso: Matematica.calcularAreaCirculo(5.0);
```

3. Métodos Construtores

```
public class Livro {  
    String titulo;  
    String autor;  
  
    // Construtor  
    public Livro(String titulo, String autor) {  
        this.titulo = titulo;  
        this.autor = autor;  
    }  
}
```

4. Métodos Getters e Setters

```
public class Produto {
    private String nome;
    private double preco;

    // Getter
    public String getNome() {
        return nome;
    }

    // Setter
    public void setNome(String nome) {
        this.nome = nome;
    }

    public double getPreco() {
        return preco;
    }

    public void setPreco(double preco) {
        if (preco >= 0) {
            this.preco = preco;
        }
    }
}
```

Métodos com Parâmetros e Retorno

```
public class Retangulo {
    private double largura;
    private double altura;

    // Método sem retorno (void)
    public void definirDimensoes(double largura, double altura) {
        this.largura = largura;
        this.altura = altura;
    }

    // Método com retorno
    public double calcularArea() {
        return largura * altura;
    }

    // Método com retorno String
    public String obterInformacoes() {
        return "Largura: " + largura + ", Altura: " + altura;
    }
}
```

Construtores

O que são Construtores?

- Métodos especiais para **inicializar objetos**
- Têm o **mesmo nome** da classe
- **Não têm tipo de retorno**
- São chamados com `new`

Tipos de Construtores

1. Construtor Padrão

```
public class Aluno {  
    String nome;  
    int idade;  
  
    // Construtor padrão (fornecido pelo Java se não definido)  
    public Aluno() {  
        // Inicializa com valores padrão  
    }  
}
```

2. Construtor com Parâmetros

```
public class Aluno {  
    String nome;  
    int idade;  
  
    // Construtor com parâmetros  
    public Aluno(String nome, int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
}
```

3. Sobrecarga de Construtores

```
public class Aluno {  
    String nome;  
    int idade;  
    String curso;  
  
    // Construtor 1  
    public Aluno() {  
        this.nome = "Não definido";  
        this.idade = 0;  
    }  
}
```

```
        this.curso = "Não definido";
    }

    // Construtor 2
    public Aluno(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
        this.curso = "Não definido";
    }

    // Construtor 3
    public Aluno(String nome, int idade, String curso) {
        this.nome = nome;
        this.idade = idade;
        this.curso = curso;
    }
}
```

Modificadores de Acesso

Visibilidade dos Membros da Classe

Modificador	Classe	Pacote	Subclasse	Todos
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default	✓	✓	✗	✗
private	✓	✗	✗	✗

Exemplos

```
public class ExemploAcesso {
    public int publico;           // Acesso de qualquer lugar
    protected int protegido;     // Acesso no pacote e subclasses
    int pacote;                  // Acesso apenas no pacote (default)
    private int privado;         // Acesso apenas na classe

    // Método público para acessar atributo privado
    public int getPrivado() {
        return this.privado;
    }

    public void setPrivado(int valor) {
        this.privado = valor;
    }
}
```

Encapsulamento

Princípio do Encapsulamento

"Esconder a implementação e expor apenas interfaces seguras"

Benefícios

- **Controle de acesso** aos dados
- **Validação** nos setters
- **Flexibilidade** para mudar implementação
- **Manutenibilidade** do código

Implementação com Getters e Setters

```
public class ContaBancaria {
    // Atributos privados (encapsulados)
    private String numeroConta;
    private double saldo;
    private String titular;

    // Construtor
    public ContaBancaria(String numeroConta, String titular) {
        this.numeroConta = numeroConta;
        this.titular = titular;
        this.saldo = 0.0;
    }

    // Getters
    public String getNumeroConta() {
        return numeroConta;
    }

    public double getSaldo() {
        return saldo;
    }

    public String getTitular() {
        return titular;
    }

    // Setters com validação
    public void setTitular(String titular) {
        if (titular != null && !titular.trim().isEmpty()) {
            this.titular = titular;
        }
    }

    // Métodos específicos da classe
    public void depositar(double valor) {
        if (valor > 0) {
            saldo += valor;
            System.out.println("Depósito realizado: R$ " + valor);
        }
    }
}
```



```
        } else {
            System.out.println("Valor de depósito inválido!");
        }
    }

    public boolean sacar(double valor) {
        if (valor > 0 && valor <= saldo) {
            saldo -= valor;
            System.out.println("Saque realizado: R$ " + valor);
            return true;
        } else {
            System.out.println("Saldo insuficiente ou valor inválido!");
            return false;
        }
    }
}
```

Exemplos Práticos

Exemplo 1: Classe Pessoa Completa

```
public class Pessoa {
    // Atributos
    private String nome;
    private int idade;
    private String cpf;
    private double altura;
    private static int totalPessoas = 0;

    // Construtores
    public Pessoa() {
        totalPessoas++;
    }

    public Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
        totalPessoas++;
    }

    public Pessoa(String nome, int idade, String cpf, double altura) {
        this.nome = nome;
        this.idade = idade;
        this.cpf = cpf;
        this.altura = altura;
        totalPessoas++;
    }

    // Getters e Setters
    public String getNome() {
        return nome;
    }
}
```

```
}

public void setName(String nome) {
    if (nome != null && nome.length() >= 2) {
        this.nome = nome;
    }
}

public int getIdade() {
    return idade;
}

public void setIdade(int idade) {
    if (idade >= 0 && idade <= 150) {
        this.idade = idade;
    }
}

public String getCpf() {
    return cpf;
}

public void setCpf(String cpf) {
    if (cpf != null && cpf.length() == 11) {
        this.cpf = cpf;
    }
}

public double getAltura() {
    return altura;
}

public void setAltura(double altura) {
    if (altura > 0) {
        this.altura = altura;
    }
}

public static int getTotalPessoas() {
    return totalPessoas;
}

// Métodos específicos
public void fazerAniversario() {
    this.idade++;
    System.out.println("Feliz aniversário, " + nome + "! Agora você tem " +
idade + " anos.");
}

public String obterInformacoes() {
    return "Nome: " + nome +
        ", Idade: " + idade +
        ", CPF: " + cpf +
        ", Altura: " + altura + "m";
}
```

```
// Método estático
public static boolean validarCPF(String cpf) {
    return cpf != null && cpf.length() == 11 && cpf.matches("\\d+");
}
}
```

Exemplo 2: Classe Círculo com Cálculos

```
public class Circulo {
    // Atributos
    private double raio;
    private String cor;
    private static final double PI = 3.14159;

    // Construtores
    public Circulo() {
        this.raio = 1.0;
        this.cor = "vermelho";
    }

    public Circulo(double raio) {
        this.raio = raio;
        this.cor = "vermelho";
    }

    public Circulo(double raio, String cor) {
        this.raio = raio;
        this.cor = cor;
    }

    // Getters e Setters
    public double getRaio() {
        return raio;
    }

    public void setRaio(double raio) {
        if (raio > 0) {
            this.raio = raio;
        }
    }

    public String getCor() {
        return cor;
    }

    public void setCor(String cor) {
        if (cor != null && !cor.trim().isEmpty()) {
            this.cor = cor;
        }
    }
}
```

```
// Métodos de cálculo
public double calcularArea() {
    return PI * raio * raio;
}

public double calcularCircunferencia() {
    return 2 * PI * raio;
}

public double calcularDiametro() {
    return 2 * raio;
}

// Método para exibir informações
public void exibirInformacoes() {
    System.out.println("Círculo: " +
        "Raio = " + raio +
        ", Cor = " + cor +
        ", Área = " + String.format("%.2f", calcularArea()) +
        ", Circunferência = " + String.format("%.2f",
calcularCircunferencia()));
}
}
```

Exemplo 3: Programa Principal para Testar as Classes

```
public class TesteClasses {
    public static void main(String[] args) {
        // Testando a classe Pessoa
        System.out.println("=== TESTE CLASSE PESSOA ===");
        Pessoa pessoa1 = new Pessoa("João Silva", 25, "12345678901", 1.75);
        Pessoa pessoa2 = new Pessoa("Maria Santos", 30);

        System.out.println(pessoa1.obterInformacoes());
        System.out.println(pessoa2.obterInformacoes());

        pessoa1.fazerAniversario();
        pessoa1.setAltura(1.76);

        System.out.println("Total de pessoas: " + Pessoa.getTotalPessoas());

        // Testando a classe Circulo
        System.out.println("\n=== TESTE CLASSE CÍRCULO ===");
        Circulo circulo1 = new Circulo();
        Circulo circulo2 = new Circulo(5.0, "azul");

        circulo1.exibirInformacoes();
        circulo2.exibirInformacoes();

        circulo1.setRaio(3.0);
        circulo1.setCor("verde");
        circulo1.exibirInformacoes();
    }
}
```

```
// Testando a classe ContaBancaria
System.out.println("\n=== TESTE CLASSE CONTA BANCÁRIA ===");
ContaBancaria conta = new ContaBancaria("12345", "Carlos Oliveira");

conta.depositar(1000.0);
conta.sacar(300.0);
conta.sacar(800.0); // Deve falhar

System.out.println("Saldo final: R$ " + conta.getSaldo());
}
}
```

Exercícios

Exercício 1: Classe Retângulo

Crie uma classe **Retangulo** com:

- Atributos: **largura** e **altura** (privados)
- Construtores: padrão, com parâmetros
- Métodos: getters/setters, **calcularArea()**, **calcularPerimetro()**, **ehQuadrado()**

Exercício 2: Classe Livro

Crie uma classe **Livro** com:

- Atributos: **titulo**, **autor**, **anoPublicacao**, **preco**, **numeroPaginas**
- Métodos: getters/setters com validação, **aplicarDesconto()**, **calcularPrecoPorPagina()**

Exercício 3: Classe Aluno

Crie uma classe **Aluno** com:

- Atributos: **nome**, **matricula**, **notas** (array de 3 notas)
- Métodos: **calcularMedia()**, **situacao()** (aprovado/reprovado), **adicionarNota()**

Exercício 4: Classe Carro

Crie uma classe **Carro** com:

- Atributos: **marca**, **modelo**, **ano**, **velocidadeAtual**, **ligado**
- Métodos: **ligar()**, **desligar()**, **acelerar()**, **frear()**, **obterStatus()**

Dicas para Raciocínio

1. **Comece com analogias** (forma de bolo, planta de casa)
2. **Use exemplos do mundo real** (Carro, Pessoa, Conta Bancária)
3. **Enfatize o encapsulamento** desde o início
4. **Mostre a evolução**: comece sem encapsulamento, depois adicione

5. **Pratique bastante** com exercícios graduais
6. **Use diagramas UML** para visualizar classes

Boas Práticas

1. **Nomes descritivos** para classes, atributos e métodos
2. **Sempre use encapsulamento** (atributos privados)
3. **Documente com comentários**
4. **Valide dados** nos setters
5. **Mantenha coesão** (cada classe com responsabilidade única)

```
// Exemplo de boa prática
/**
 * Representa um produto em um sistema de e-commerce
 */
public class Produto {
    private String codigo;
    private String nome;
    private double preco;
    private int estoque;

    // Construtores, getters, setters e métodos específicos...
}
```